VISTA: Verifier-in-the-Loop Agentic RL for Semantic Program Synthesis in Quantum Computing

Submission ID

Abstract

We present VISTA, a verifier-in-the-loop, tool-augmented agentic RL system for semantic program synthesis, with OpenQASM 3.0 quantum circuit generation as a successful application. Our core idea is verifier-in-the-loop post-training: an external quantum verification toolchain evaluates complete generated programs and returns training signals grounded in execution semantics rather than text plausibility. To make this practical and stable, VISTA introduces two optimization techniques: (i) hierarchical verified reward optimization, which stages learning from executability to distributional alignment, objective-level semantic quality, and optimization utility; and (ii) cost-aware gated evaluation optimization, which prioritizes low-cost checks and invokes expensive simulator/optimizer evaluations only for promising candidates. Across a broad suite of quantum optimization tasks, VISTA improves both executability and semantic quality over prompting-based industrial LLMs and SFT-only and RL-only post-training baselines. These results show that verifier-in-the-loop agentic RL, combined with hierarchical reward shaping and gated evaluation, is an effective design pattern for AI agent systems in domains with expensive semantic verification, with quantum circuit generation as a successful application.

ACM Reference Format:
Submission ID. 2026. VISTA: Verifier-in-the-Loop Agentic RL for Semantic Program Synthesis in Quantum Computing. In *Proceedings of ACM Conference on AI and Agentic Systems (CAIS '26).* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Agentic systems increasingly generate executable artifacts—code, configurations, and domain-specific programs—that must act through external toolchains. In many real deployments, the deciding factor is not whether an output looks plausible, but whether it satisfies *semantic* requirements that only tools can validate. In these settings, correctness cannot be judged from text alone: it must be *executed* and checked by a compiler, simulator, solver, or optimizer. Prompt-only generation therefore creates a recurring gap between *plausibility* and *deployability*: outputs that appear reasonable often fail under verification, either as hard failures (non-compiling artifacts) or soft failures (compiling artifacts with semantically misaligned behavior). Closing this generation–verification gap is a core challenge for agentic systems operating under tool-defined semantics.

A practical solution must satisfy three requirements simultaneously. It must be *stable*: learning should not collapse under sparse, delayed feedback. It must be *efficient*: verification cannot be run indiscriminately when each check is expensive. And it must preserve *semantic fidelity*: improvements should reflect true task semantics rather than superficial proxies. These requirements mirror what makes industrial coding agents successful: they wrap LLM generation with build/test sandboxes and iterate on failures. However, that workflow implicitly assumes that semantic checks (compile/tests) are relatively cheap and decisive; our target setting breaks this assumption.

Two properties make expensive-semantics synthesis fundamentally a *systems* problem. First, semantic verification is typically *multi-stage* with heterogeneous costs: artifacts can fail early (syntax/constraints) or only after expensive evaluation (objective/utility). Second, semantic feedback is *costly* and *black-box*: correctness may require simulation, search, or optimization, and small numerical errors can destroy utility. As a result, simply "running the verifier everywhere" is infeasible, while naive reinforcement learning becomes unstable under sparse, high-variance, costly rewards.

Recent state-of-the-art systems move beyond text-only generation by incorporating *execution feedback* or *environment feedback* into the synthesis loop—for example, using unit tests as training signals for code generation with reinforcement learning [24] or using agentic feedback and memory to improve decisions from trial-and-error [37]. These approaches substantially improve *functional correctness* when semantic checks are cheap (e.g., compile-and-test). However, they do not directly resolve domains where semantic verification itself dominates cost (e.g., requiring simulation and optimization) and where correctness is highly parameter-sensitive. In this regime, the bottleneck is not how to obtain *some* semantic signal, but how to (i) make learning stable under verifier-defined semantics and (ii) allocate verification cost effectively.

This paper asks: *How can we build verifier-in-the-loop agentic synthesis systems that are simultaneously stable, efficient, and semantically faithful under expensive verification?* We frame this as a design space with three concrete axes: **what** semantic signals to optimize (executability, distributional alignment, objective semantics, utility), **how** to shape these signals into learning objectives that remain stable, and **when** to spend verification budget across multi-stage toolchains. These axes align with our requirements: choosing *what* signals to optimize ensures semantic fidelity, designing *how* to shape them ensures training stability, and deciding *when* to invoke expensive verification ensures efficiency.

To address this problem, we introduce a general interface, *Verified Semantic Synthesis (VSS)*, where an instance specifies `spec/constraints/semantic` under a verification `budget` and the synthesis loop follows $x \to a \to \rho \to R$: a policy generates a candidate artifact a, an external verifier returns a structured report ρ (stage, semantic signals, costs, diagnostics), and a terminal reward R is computed from ρ. We build VISTA, a system/framework that operationalizes VSS with a verifier tool server and group-based optimization (GRPO). Quantum OpenQASM synthesis is our primary instantiation because it stresses exactly the expensive-semantics regime: verification often requires large-scale simulation on HPC/supercomputers or execution on real QPUs whose access is billed.

CAIS '26, San Jose, CA, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Quantum assembly code generation is a representative "hard-semantics" instance of VSS. OpenQASM 3.0 is a vendor-neutral, hardware-facing representation used across modern quantum software stacks, and it is particularly unforgiving: a generated program must parse, respect qubit-resource constraints, and induce measurement behavior aligned with a task objective after simulation and (often) local optimization. Achieving this requires deep domain expertise: the artifact couples discrete structure (ansatz/topology) with continuous parameters, where small numeric errors can destroy objective quality even when the program compiles. Verification is also expensive in practice: IBM Quantum's pay-as-you-go access starts at **$96 per minute** of QPU usage [2], and AWS Braket charges a per-task fee plus per-shot fees on real QPUs (e.g., IonQ Aria **$0.30 per task + $0.03 per shot**); reservation mode access is even higher (e.g., IonQ Aria at **$7,000 per hour**) [1]. These costs make "verify everything" an unrealistic baseline and motivate cost-aware verifier scheduling. In our setting, LLM outputs commonly fail in distinct modes: (i) compilation failure, (ii) qubit-count mismatch, or (iii) compilable but task-misaligned circuits with poor parameter values (Fig. **??**).

We make the following contributions.

(1) VSS interface for verifier-defined semantics. We introduce *Verified Semantic Synthesis (VSS)*, an interface for agentic synthesis where artifact quality is defined by an external verifier report and a terminal reward under an explicit verification budget (§2).

(2) Hierarchical verified reward optimization over VSS signals. We propose a staged reward design that converts multi-stage verification outcomes into stable learning signals, optimizing generation from executability to semantic alignment and deployment-oriented utility (§4).

(3) Cost-aware gated evaluation optimization for VSS verifiers. We design a gated verifier execution policy that schedules expensive verification (simulation/optimization) only for promising candidates, improving training efficiency while preserving semantic fidelity (§5).

Evaluation highlights (§7). On a broad suite of quantum optimization tasks, VISTA achieves near-perfect executability (SCR **99.31%** Pass@1 and **100%** Pass@10) and improves semantic quality (e.g., HQCR **17.24%** Pass@1 and **27.24%** Pass@10), outperforming prompting-based industrial LLMs and SFT-only/RL-only baselines; we further validate each optimization via ablations, study scaling with circuit size, and demonstrate deployment on real quantum hardware.

2 Problem and Application Context

[TODO: SQUEEZE: Keep this section to problem formulation + why quantum/OpenQASM is a hard-semantics case. Move textbook quantum/background content to Appendix.]

Semantic program synthesis with external verification. We study program synthesis settings where correctness is *semantic* and can only be reliably assessed by running an external verifier toolchain (e.g., compiler, simulator, and task evaluator), rather than by text plausibility. This paper uses quantum program synthesis as a concrete, high-stakes instance of this class of problems.

2.1 Target Representation: OpenQASM

OpenQASM (Open Quantum Assembly Language) [8] is a low-level programming language designed for expressing quantum circuits and operations, which is similar to traditional Hardware Description Language (HDL) like Verilog and VHDL. It serves as an intermediate representation (IR) for quantum algorithms, allowing them to be executed on various quantum hardware platforms. OpenQASM provides a standardized way to describe quantum gates, measurements, and other operations, making it easier for developers to write and share quantum programs.

OpenQASM is widely used in the quantum computing community, since many popular quantum computing software frameworks, such as IBM's Qiskit [19], Google's Cirq [10], Microsoft's QDK [30], and Rigetti's Forest[38], support OpenQASM as a means of serializing quantum circuits in a standard way. Developers to write quantum algorithms in a high-level language and then compile them down to OpenQASM for distribution between different quantum hardware backends. OpenQASM has become the conjunction between quantum software and hardware, being critical for both platform-agnostic and platform-specific optimization, mapping, scheduling, evaluation, profiling, and simulation.

2.2 Quantum Application Context: Optimization Circuits

Quantum computing exploits the principles of quantum mechanics to perform computations using quantum bits (qubits) instead of classical bits. A qubit is a unit vector in a two-dimensional Hilbert space and can exist in a superposition $\alpha|0\rangle + \beta|1\rangle$ with $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$ [32]. Multi-qubit states arise via tensor products, and computation proceeds by applying unitary gates. Rotation gates $R_x(\theta),\ R_y(\theta),\ R_z(\theta)$ are unitary for all θ, and together with entangling operations (e.g., CNOT) and standard gates (e.g., Hadamard, CZ), they form a universal gate set [43]. By combining these gates into parameterized circuits $U(\theta)$, one obtains a flexible framework for variational quantum algorithms.

One of the most promising applications is quantum optimization [3]. Classical problems such as QUBO and HUBO [6, 29] can be rewritten in terms of spin variables and mapped to Pauli-Z operators, turning them into Hamiltonian minimization tasks. The goal is to prepare a parameterized state $U(\theta)|0\rangle$ whose expectation value $\langle 0|U^\dagger(\theta)HU(\theta)|0\rangle$ approximates the ground state of H. Hybrid quantum-classical methods iteratively optimize θ: QAOA uses a cost Hamiltonian and a mixer ansatz [13], VQE employs expressive or hardware-efficient ansatzes [34], and adaptive VQE constructs circuits gate by gate from a predefined pool [15].

Verified Semantic Synthesis (VSS).. We model our setting as *verified semantic synthesis*, where artifact quality is defined by an external toolchain rather than text plausibility. A synthesis instance is a tuple $x \triangleq (\texttt{spec}, \texttt{constraints}, \texttt{semantics}, \texttt{budget})$: (i) `spec` is the natural-language specification, (ii) `constraints` are machine-checkable requirements (e.g., resource limits), (iii) `semantics` specifies how to evaluate task behavior (e.g., an objective function or reference behavior), and (iv) `budget` bounds verification cost (time/-calls/shots/steps). Given x, a policy π_θ produces a *candidate artifact* a (a complete artifact representation plus optional metadata), i.e.,

$a \sim \pi_\theta(\cdot \mid x)$. An external verifier V then evaluates (x, a) and returns a *verifier report* $\rho \triangleq (\texttt{stage}, \texttt{signals}, \texttt{cost}, \texttt{diagnostics})$, where `signals` are semantic measurements, `cost` accounts verifier effort, and `diagnostics` captures failure reasons (e.g., compiler errors). Learning optimizes π_θ against a terminal reward computed from the report, $R(x, a) \triangleq R(\rho)$, rather than against text likelihood.

Instantiation in this paper. We instantiate VSS for OpenQASM 3.0 quantum circuit synthesis: x contains a natural-language task description, qubit/resource constraints, a problem-specific semantic descriptor (e.g., a Hamiltonian and evaluation procedure), and verification budgets. The verifier V is a toolchain (compiler/parser + simulator + task evaluator + optional local optimizer) that produces stage-wise signals such as syntactic validity, distributional alignment, Hamiltonian expectation value, and optimization progress; VISTA implements two optimizations on top of this abstraction: (i) a hierarchical verified reward that maps `signals` to $R(\rho)$, and (ii) a cost-aware gated evaluation policy that chooses which verification stages to run under `budget`.

3 VISTA System Overview

[TODO: SQUEEZE: Keep only the system overview + one pipeline figure in main. Move protocol/prompt format details to Appendix.]

High-level architecture. VISTA couples an LLM actor with a verifier tool server (compiler, simulator, and task evaluator) and optimizes the actor via group-based reinforcement learning on verified terminal rewards. Figure 1 gives the end-to-end pipeline.

3.1 Agentic RL with Tool-Augmented Verification

Agentic Reinforcement Learning with Tool Use (ARLT) enhances domain-specific performance in the LLM post-training stage by allowing the model to interact with external tools and learn from verified feedback. In this setting, the policy LLM π_θ generates responses token by token, where τ_t denotes the token at step t and $\tau_{<t}$ denotes the preceding context. A complete generated response $\tau = (\tau_1, \ldots, \tau_T)$, corresponding to a full program or solution, is evaluated by a verified reward function $R(\tau)$.

Group Relative Policy Optimization (GRPO) [36] is a widely used RL algorithm in recent ARLT systems [21, 35], and has demonstrated strong empirical performance since [9]. Given a group of G rollout trajectories $\{\tau_i\}_{i=1}^G$ sampled from the current policy, GRPO optimizes the following clipped surrogate objective:

$$\mathcal{J}(\theta) = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\tau_i|}\sum_{t=1}^{|\tau_i|}\min\Big(r_t^i(\theta)\,\hat{A}_i,\ \mathrm{clip}\big(r_t^i(\theta), 1-\epsilon, 1+\epsilon\big)\,\hat{A}_i\Big), \tag{1}$$

where

$$r_t^i(\theta) = \frac{\pi_\theta(\tau_t^i \mid \tau_{<t}^i)}{\pi_{\mathrm{ref}}(\tau_t^i \mid \tau_{<t}^i)}$$

is the token-level importance ratio with respect to a reference policy π_{ref}. The advantage $\hat{A}_i$ is computed via group-wise normalization:

$$\hat{A}_i = \frac{R(\tau_i) - \mu}{\sigma}, \qquad \mu = \frac{1}{G}\sum_{j=1}^{G} R(\tau_j),\ \sigma = \mathrm{std}\big(\{R(\tau_j)\}_{j=1}^G\big).$$

In our setting, rewards are terminal—i.e., computed once after a complete OpenQASM program is generated rather than at intermediate token steps—and the same advantage $\hat{A}_i$ is assigned to all tokens within trajectory τ_i.

3.2 End-to-End Verified Training Pipeline

End-to-End Workflow. Figure 1 illustrates the overall design of VISTA, which treats natural-language problem descriptions as high-level specifications and translates them into executable quantum circuits in OpenQASM through verified reinforcement learning. Our training framework is built on Verl [42] and therefore follows its prompt structure. Each training input consists of two components: (1) a system message that defines the model's role as a quantum circuit design assistant and provides instructions for generating OpenQASM circuits for quantum optimization tasks; and (2) a user message that specifies the problem description, the number of qubits, and any additional constraints.

During training, (A) the LLM actor receives the input prompt and generates a batch of OpenQASM circuits; (B) the generated circuits are encapsulated into a batch of tool calls; (C) these tool calls are dispatched to the Quantum Tool Server for execution and reward evaluation; (D) the server simulates each circuit, evaluates its performance on the target quantum task, and computes rewards according to predefined criteria; and (E) the rewards are processed through group-based computations to obtain advantages, which are then used to update the LLM actor via the GRPO algorithm. In our setup, a *group* consists of multiple quantum circuit samples generated from the same input prompt, enabling VISTA to compare the relative performance of alternative solutions to the same quantum task.

Online Interaction with a Quantum Agent. Building on Jiang et al. [21], our quantum tool server (quantum agent) consolidates environment interaction and reward computation into a single tool module (see Figure 1). At each RL step (C), the quantum agent receives a batch of tool calls via HTTP and executes specialized reward calculations for the generated quantum circuits.

4 Optimization Technique I: Hierarchical Verified Reward

[TODO: SQUEEZE: Keep the reward hierarchy definition and one paragraph of rationale in main. Move derivations, extra theoretical motivation, and extended notation to Appendix.]

4.1 Hierarchical Verified Reward Design

Reward rationale and theoretical motivation. For quantum optimization tasks, the objective is to produce a circuit whose measurement distribution concentrates on low-energy bitstrings, equivalently minimizing the expected energy:

$$J(C) := \langle\psi_C|H|\psi_C\rangle = \sum_z p_C(z)\,E(z).$$

Here, C denotes a quantum circuit that prepares the state $|\psi_C\rangle = U_C|0\rangle^{\otimes n}$, and $p_C(z) = |\langle z|\psi_C\rangle|^2$ is the distribution obtained by measuring $|\psi_C\rangle$ in the computational basis. For the optimization problems considered in this work (e.g., QUBO and Ising-type graph problems), the cost Hamiltonian H is diagonal in the computational

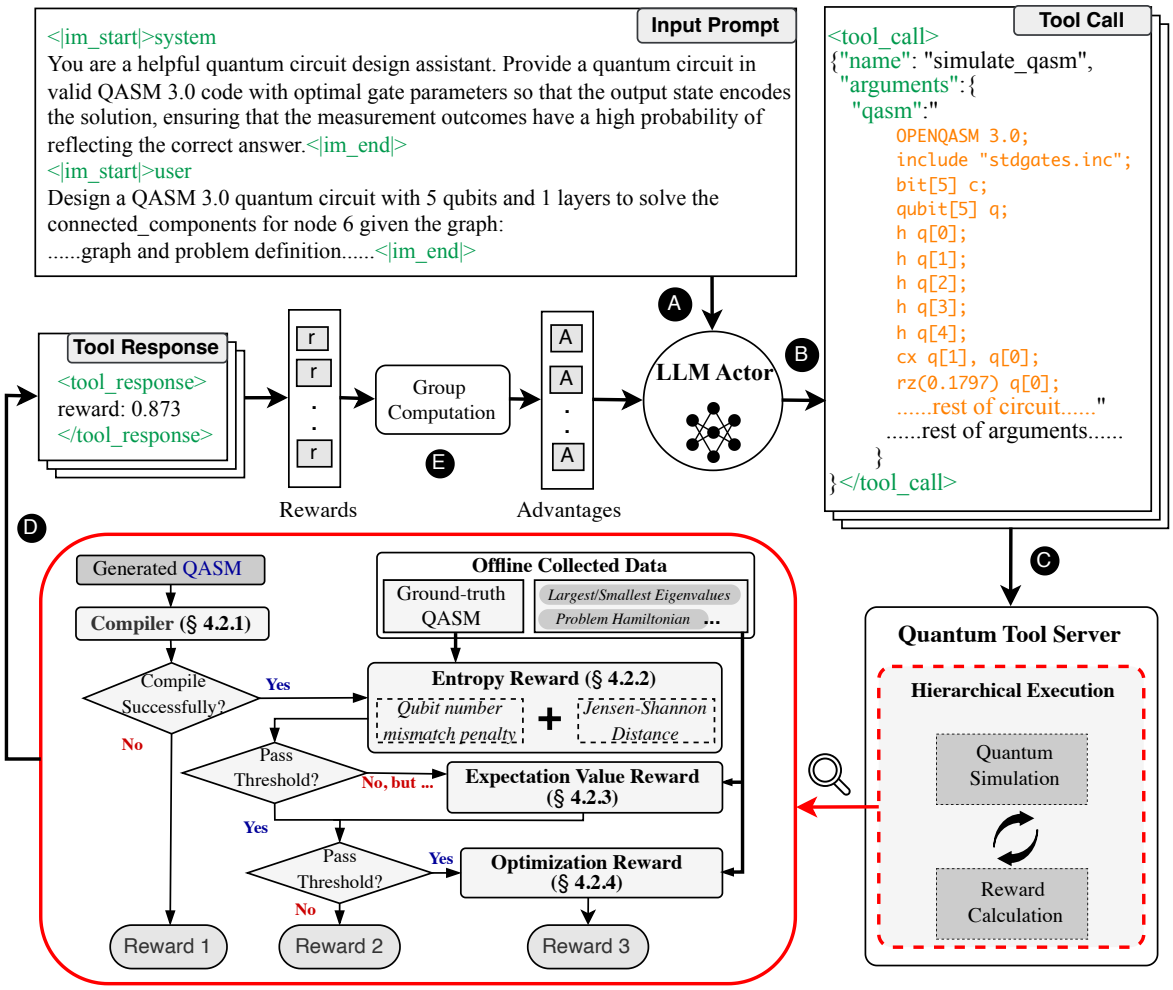


Fig. 1: Vista design overview.

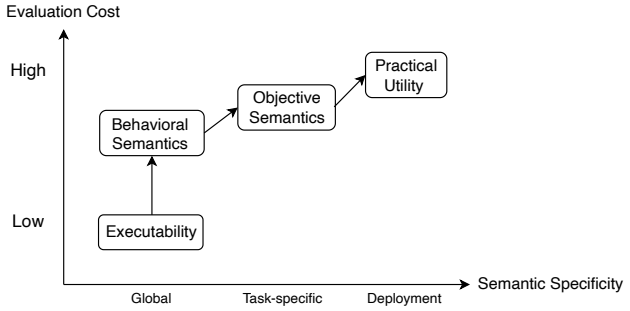


Fig. 2: Reward design space for quantum circuit generation. The semantic specificity axis reflects increasing contextual grounding of evaluation signals, from global behavior checks, through Hamiltonian-defined task objectives, to deployment-oriented algorithmic utility.

basis and can be written as

$$H = \sum_{z \in \{0,1\}^n} E(z) \, |z\rangle\langle z|,$$

where $E(z)$ is the classical objective value associated with bitstring z.

Lemma 1 (Distributional alignment bounds energy error). Let p_C and $p_{C^\star}$ denote the measurement distributions induced by a generated circuit C and a reference circuit $C^\star$, respectively. If H is diagonal in the computational basis, then

$$\left|J(C) - J(C^\star)\right| \le \|E\|_\infty \, \|p_C - p_{C^\star}\|_1 \le \|E\|_\infty \sqrt{2 \ln 2 \cdot \mathrm{JS}(p_C \,\|\, p_{C^\star})},$$

where $\|E\|_\infty = \max_z |E(z)|$.

Interpretation. Reducing the Jensen–Shannon divergence between measurement distributions bounds the deviation in expected energy, which motivates distributional alignment as a low-variance surrogate for the optimization objective.

Formal reward definition. For each generated trajectory τ_i, corresponding to a complete OpenQASM program, we define a scalar terminal reward $R(\tau_i)$ as

$$R(\tau_i) = \begin{cases} -1, & R_{\text{syn}}(\tau_i) = -1, \\ w_2\, s_2(\tau_i), & R_{\text{syn}}(\tau_i) = 1 \wedge \neg\text{Prom}(\tau_i), \\ w_2\, s_2(\tau_i) + w_3\, s_3(\tau_i) + w_4\, R_{\text{opt}}(\tau_i), & R_{\text{syn}}(\tau_i) = 1 \wedge \text{Prom}(\tau_i). \end{cases} \tag{2}$$

where $R_{\text{syn}}(\tau_i) \in \{-1, 1\}$ is the syntactic validity reward, equal to 1 if the generated OpenQASM program parses successfully and -1 otherwise. The term $s_2(\tau_i)$ denotes the distributional alignment score based on the Jensen–Shannon distance, augmented with the qubit-mismatch penalty. The term $s_3(\tau_i)$ denotes the expectation-value score (defined in Equation 5), and $R_{\text{opt}}(\tau_i)$ denotes the optimization-progress reward (defined in Equation 7). The weights $w_2, w_3, w_4 \geq 0$ control the relative contribution of each reward component.

A trajectory τ_i is considered *promising* if

$$\text{Promising}(\tau_i) \iff \big(s_2(\tau_i) \geq \delta_{\text{JS}}\big) \;\vee\; \big(s_3(\tau_i) \geq \delta_{\text{EV}}\big),$$

where $\delta_{\text{JS}}, \delta_{\text{EV}} \in (0, 1)$ are predefined thresholds.

Integration with GRPO. Following the GRPO formulation introduced in §3.1, for a given input prompt x, we sample a group of G trajectories $\{\tau_i\}_{i=1}^{G}$ from the current policy $\pi_\theta(\cdot \mid x)$. Each trajectory τ_i receives a terminal reward $R(\tau_i)$ defined in Equation 2. We compute the group-normalized advantage as

$$\hat{A}_i = \frac{R(\tau_i) - \mu}{\sigma}, \qquad \mu = \frac{1}{G}\sum_{j=1}^{G} R(\tau_j), \quad \sigma = \sqrt{\frac{1}{G}\sum_{j=1}^{G}\big(R(\tau_j) - \mu\big)^2 + \varepsilon}. \tag{2}$$

The policy is updated using the GRPO clipped surrogate objective. Let $a_{i,t}$ denote the t-th token in trajectory τ_i, and define the importance ratio

$$r_{i,t}(\theta) = \frac{\pi_\theta(a_{i,t} \mid a_{i,<t}, x)}{\pi_{\text{ref}}(a_{i,t} \mid a_{i,<t}, x)}.$$

Since the reward in Equation 2 is terminal, the same advantage $\hat{A}_i$ is assigned to all tokens in τ_i. The resulting GRPO objective is

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_x\Bigg[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\tau_i|}\sum_{t=1}^{|\tau_i|} \min\Big(r_{i,t}(\theta)\hat{A}_i,\ \text{clip}(r_{i,t}(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_i\Big)\Bigg]. \tag{3}$$

In summary, the hierarchical quantum-aware reward in Equation 2 provides a verified scalar signal for each generated OpenQASM program. This signal is normalized within each prompt-level group and directly optimized via the GRPO objective in Equation 3.

Hierarchical reward mechanism. We design a hierarchical reward with four components: (i) *syntactic correctness*, (ii) *distributional alignment* (an entropy-based term), (iii) an *expectation-value* term, and (iv) an *optimization-progress* term. The computation proceeds hierarchically. We first verify syntactic correctness; if the generated OpenQASM is valid, we then measure the distributional discrepancy between the generated and ground-truth circuits (e.g., via the Jensen–Shannon distance) to quantify overall mismatch. Generated circuits may realize unitaries $\hat{U}$ that deviate substantially from the ground truth $U^\star$, leading to different measurement distributions in the computational basis. However, for a given quantum optimization task, performance can remain comparable because evaluation depends on the problem Hamiltonian H through $E(\psi) = \langle\psi|H|\psi\rangle$. To capture this task-specific behavior, we further incorporate two problem-aware terms: (a) an *expectation-value reward* that evaluates Hamiltonian-based performance, and (b) an *optimization-progress reward* that credits the improvement achieved by a local optimizer. Overall, the reward reflects both the steps required to improve the generated circuit and the gap between the converged solution and the reference.

In addition, we introduce a *qubit-mismatch penalty* in stage (ii) to discourage generating an inconsistent number of qubits. This penalty addresses a common failure mode in our training loop: the LLM may output a circuit whose qubit count differs from the target, which can lead to dimension mismatch and reward evaluation errors (e.g., generating `qubit[7] q;` for a 9-qubit prompt).

Cost-aware gated evaluation (overview). We use a cost-aware gated evaluation policy to prioritize low-cost checks and invoke expensive simulator/optimizer evaluations only for promising candidates (§5).

4.1.1 Syntactic reward. The syntactic reward captures whether the generated OpenQASM program is syntactically valid. A circuit is considered syntactically correct if it can be parsed by the Qiskit OpenQASM 3.0 parser, implying that it follows the grammar of the OpenQASM standard [8]. If parsing fails, reward computation terminates and returns -1.

4.1.2 Entropy reward. Prior work [20] evaluated circuit quality using relative entropy, i.e., Kullback–Leibler (KL) divergence. In this work, we use the Jensen–Shannon (JS) distance, which yields a bounded value in $[0, 1]$ and is more stable for RL training. The Jensen–Shannon distance is defined as

$$d_{\text{JS}}(p, q) = \sqrt{\frac{\text{JS}(p \,\|\, q)}{\log 2}}, \tag{4}$$

where

$$\text{JS}(p \,\|\, q) = \tfrac{1}{2}D_{\text{KL}}(p \,\|\, m) + \tfrac{1}{2}D_{\text{KL}}(q \,\|\, m),$$

$m = \frac{1}{2}(p + q)$, and D_{KL} is the KL divergence. We define the reward as $1 - d_{\text{JS}}(p, q) \in [0, 1]$, where 1 indicates identical distributions and 0 indicates highly dissimilar distributions. By Lemma 1, for diagonal cost Hamiltonians, reducing JS divergence also bounds the deviation in expected energy, making this term an effective surrogate for task performance.

Qubit-mismatch penalty. In practice, the generated QASM may specify a different number of qubits than the ground-truth QASM, making the entropy-based reward inapplicable due to a dimension mismatch. Let n_{gen} and n_{gt} be the qubit counts of the generated and ground-truth circuits, and let $k = \min(n_{\text{gen}}, n_{\text{gt}})$. We define a qubit-mismatch penalty

$$R_{\text{qm}} = \text{clip}_{[-0.2,0]}\big(\alpha + \beta\,\Delta n + \gamma\, a_{\text{extra}} + \eta\, e_{\text{cross}}\big), \quad \Delta n = |n_{\text{gen}} - n_{\text{gt}}|,$$

where a_{extra} counts *active* extra qubits beyond the first k (idle or reset-only ancillas incur little or no penalty), and e_{cross} counts multi-qubit gates that entangle extra wires with core wires. To ensure fairness, we compute d_{JS} on the first k qubits (via marginalization) and down-scale its weight by the mismatch severity.

For the expectation-value and optimization-progress terms below, we pad the problem Hamiltonian with identities so that its width matches the circuit, preserving comparability while preventing reward hacking through ancillary qubits.

4.1.3 Expectation-value reward. While distributional alignment controls energy error relative to a specific reference distribution, multiple optimal or near-optimal solutions may induce different distributions. The expectation-value reward therefore directly measures normalized suboptimality with respect to the Hamiltonian.

Our training dataset consists of optimization problems expressed as eigenvalue minimization tasks. For an ideal circuit, measurement would return the optimal eigenvalue with probability 1, and the expectation value would coincide with the optimum. In practice, the measured expectation value is typically larger. We therefore define a reward based on the expectation value of the cost Hamiltonian.

For a syntactically correct circuit, we simulate the circuit and compute the expectation value of the problem-specific cost Hamiltonian, denoted by E_{gen}. We also compute the minimum and maximum eigenvalues of the Hamiltonian, denoted by $E_{\min}$ and $E_{\max}$, respectively. Since $0 \leq E_{\min} < E_{\text{gen}} < E_{\max}$, we apply min–max normalization:

$$f_{\max}^{\min}(E_{\text{gen}}) := \frac{E_{\text{gen}} - E_{\min}}{E_{\max} - E_{\min}}, \tag{5}$$

which yields a value in $[0, 1]$. We define the expectation-value reward as

$$s_3(\tau_i) := 1 - f_{\max}^{\min}(E_{\text{gen}}) = 1 - \frac{E_{\text{gen}} - E_{\min}}{E_{\max} - E_{\min}}, \tag{6}$$

4.1.4 Optimization reward. It is unlikely that LLMs generate parameterized quantum circuits that immediately solve the given optimization problems. A practical workflow therefore includes an additional phase where the user continues optimizing the parameters of the generated circuit. Given the difficulty of parameter optimization, a key indicator of usefulness is the number of optimization steps required to reach a sufficiently low expectation value.

We define an optimization-progress reward as $1/(1+n)$, where n is the number of optimization steps required for the generated circuit to reach an optimized circuit for the given problem. In addition, we favor circuits that achieve lower final expectation values. Let E_{opt} denote the expectation value after local optimization, and let $E_{\min}$ again be the optimal eigenvalue. We therefore use

$$R_{\text{opt}}(\tau_i) := \frac{1}{1+n} + \left(1 - f_{\max}^{\min}(E_{\text{opt}})\right), \tag{7}$$

where $f_{\max}^{\min}$ is defined in Equation 5. Under standard local convergence assumptions of classical optimizers, the number of optimization steps required to reach a target energy threshold is monotone in the initial suboptimality, making optimization progress a proxy for initialization quality.

5 Optimization Technique II: Cost-Aware Gated Evaluation

[TODO: SQUEEZE: Keep gating policy and cost/benefit intuition in main. Quantify savings in main with one table/plot; move deeper breakdown to Appendix.]

Motivation. Semantic verification is expensive (simulation and, optionally, downstream optimization). VISTA therefore schedules verifier work to avoid wasting compute on clearly low-quality candidates.

Gated evaluation policy. We adopt a gated procedure to compute rewards, as illustrated in the red line box of Figure 1. If the distributional divergence between the generated and ground-truth circuits exceeds a threshold, we additionally evaluate the expectation-value reward. If either the (normalized) expectation-value score surpasses its threshold or the distributional divergence falls below its threshold, we run a local optimizer and assign an optimization-progress reward. This design avoids expensive evaluations for clearly low-quality circuits while remaining robust to cases where distributions differ but task performance is comparable. In the following, we introduce each reward component in detail.

[TODO: Quantify verifier-cost savings from gated evaluation: stage pass rates (syntax → JS → EV → Opt), simulator/optimizer invocations avoided, and end-to-end throughput gains; include a no-gating variant under matched quality or matched compute.]

6 Implementation

[TODO: SQUEEZE: Keep a short implementation summary in main (tool server, batching). Move full hyperparams, deployment knobs, and engineering details to Appendix.]

Tool server and execution. VISTA implements environment interaction and reward computation as a tool server that receives batched tool calls and executes compilation, simulation, and task evaluation. The system is built on Verl [42] and follows its prompt structure; details of training setup and hyperparameters are provided in subsection A.1.

[TODO: (Optional) Add concrete batching/caching details for the verifier server (request format, batch size, parallelism) and report its throughput/latency under representative load.]

7 Evaluation

[TODO: SQUEEZE (9-page main): Keep only (i) key metrics + main results table, (ii) 3-5 sentence takeaway, (iii) ablation table with 3-4 bullet takeaways, (iv) one scalability figure, and (v) 1/2-page real-device summary. Move full training setup, detailed baseline descriptions, threshold sweeps, and latency breakdown to Appendix.]

7.1 Experimental Setup and Metrics

7.1.1 Training Setup. **[TODO: SQUEEZE: Move dataset construction details + splits + references to Appendix (keep 1–2 summary sentences in main).]**

Training data and references. We build on the AGENT-Q dataset [20], which contains 13,914 OpenQASM circuits spanning 12 graph-optimization primitives (QAOA/VQE/adaptive VQE) with offline-optimized parameters and associated Hamiltonian metadata. These offline circuits serve as reward references for distributional and objective-based verification. Detailed dataset statistics and the ground-truth construction procedure are deferred to Appendix A.1.

[TODO: SQUEEZE: Move full training hyperparameters, rollout settings, and infrastructure details to Appendix; keep only model size, RL algorithm, and hardware summary in main.]

Model and training. We post-train a 4B Qwen3-based model using GRPO with tool-augmented verification on 16×H100-64GB GPUs (vLLM rollouts, temperature 0.7, top-p 0.8). The base checkpoint is obtained via supervised fine-tuning following Agent-Q [20]. Full hyperparameters are in Appendix A.1.

[TODO: SQUEEZE: Keep metric definitions brief in main; move thresholds and additional metric variants to Appendix.]

Evaluation Metrics. We evaluate the fine-tuned model across four complementary metrics designed to assess both syntactic fidelity and optimization quality. First, we measure ***syntactical correctness ratio (SCR)***, defined as the percentage of generated outputs that can be parsed as valid OpenQASM 3.0 circuits using Qiskit's QASM parser. This metric captures whether the model has internalized the grammar of the domain-specific language. Second, we perform ***successful rate of expectation value (SREV)***, where each syntactically correct circuit is simulated and the expectation value for the problem-specific cost Hamiltonian H is computed. We compute the expectation value of generated circuit C as $E(C) = \langle\psi_C|H|\psi_C\rangle$. Let $E^\star$ denote the ground-truth QASM's expectation. The circuit is counted as *successful* if $|E(C) - E^\star| \leq 0.2$, and SREV is the percentage of successful QASMs. Third, we evaluate the ***relative entropy (RE) of probability distributions*** by computing the KL divergence $D_{\mathrm{KL}}(P_{\mathrm{sol}} \,\|\, P_{\mathrm{gen}})$ between the outcome distribution of the generated circuit and that of the optimized reference. Finally, we compute the ***High-Quality Circuit Ratio (HQCR)***, defined as the proportion of generated circuits whose relative entropy against the ground-truth distributions is within a threshold 0.1. This metric provides a more interpretable measure of how often the model produces reasonable solutions. Each metric is measured in Pass@1 and Pass@10, where Pass@1 evaluates whether a single sampled QASM per prompt meets the criterion, and Pass@10 evaluates whether at least one out of ten sampled QASMs meets the criterion. Results are shown in Table 1, where up/down arrows by column names indicate whether higher or lower values are better.

[TODO: SQUEEZE: Keep only baseline categories + 1-line description in main; move full prompting setup/few-shot examples and any extra baselines to Appendix.]

Baselines. The baselines can be categorized into three groups. **(i) *Prompting-Based State-of-the-art LLMs*:** We evaluate DeepSeek-V3 [27] and OpenAI's GPT-4o [17] and GPT-5 [33] via their official API, with GPT-5 being OpenAI's latest flagship. All models are evaluated using few-shot prompting with four demonstration examples. **(ii) *SFT-only*:** We train Qwen-3-4B using SFT only. **(iii) *RL-only*:** We evaluate a cold-start model trained using the GRPO algorithm [36].

7.2 End-to-End Effectiveness vs. Baselines

Performance Analysis. We report the performance of VISTA in Table 1. VISTA achieves strong results against all baselines, with a pass@1 of 99.31% SCR. Compared to the best competing methods, it yields a +12.95% improvement in SREV (22.41 vs. 19.41 for Cold Start (GRPO)), an +8.87% reduction in RE (11.61 vs. 12.74 for Agent-Q), and a +13.64% gain in HQCR over Agent-Q. For pass@10, we even achieve 100% syntactical correctness and even better semantic improvement.

Meanwhile, as noted earlier, HQCR was measured with a fixed threshold of 0.1. In Figure 3a, we vary this threshold from 0.1 to 0.9 and report The fraction of valid QASMs that qualify as high-quality. At best, VISTA achieves a +13.65% improvement over Agent-Q, +72.4% over DeepSeek-V3, and up to 1.65× and 1.50× improvements over GPT-4o and GPT-5, respectively. Furthermore, to quantify the semantic closeness of the generated QASMs to the ground truth, we also measure $\Delta E = |E(C) - E^\star|$ defined before; note that smaller values indicate higher-quality QASMs. Figure 3b illustrates the distribution of ΔE for each model, VISTA achieves substantial improvements over all baselines, including a 4.9% reduction in the median ΔE and a 9.7% reduction in the upper quartile compared to the second-best Agent-Q.

In addition, we compare VISTA-generated QASMs with a *random parameter initialization baseline*, a common practice in hybrid quantum-classical algorithms. For each VISTA circuit, we evaluated JS-divergence and expectation value relative to the Hamiltonian and ground truth, alongside 100 randomized variants with parameters sampled from $(-\pi, \pi]$. The results show that VISTA consistently outperforms random initialization, achieving lower JS-divergence (0.79 vs. 0.95) and expectation values closer to the optimum (0.16 vs. 0.36).

[TODO: TODO (Missing experiment): Per-primitive breakdown. Add a compact per-task/primitive breakdown (e.g., HQCR or SREV) to show improvements are not driven by a subset of tasks. Put the full 12-primitive table/plot in the appendix, and summarize key takeaways in the main text.]

7.3 Which Design Choices Matter? Reward and Gating Ablations

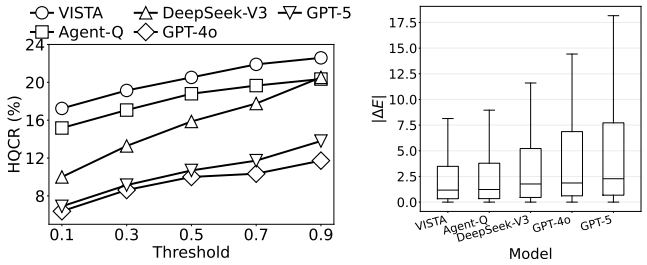
We quantify the contribution of each reward term by independently removing each component of the hierarchy reward in §4. For each variant, we disable exactly one component while keeping the remainder unchanged; we also report a *Validity-only* sanity baseline, which only returns reward based on whether the generated QASM is valid(1) or not(−1). All runs are initialized from the same SFT checkpoint and trained with matched compute (identical optimizer, schedule, batch size, steps, and decoding settings). The evaluation metrics are the same as presented in §4, and the results are summarized in Table 2.

Distributional alignment (RE) is the primary driver of all metrics. Removing RE causes the largest drop across semantic metrics and, unexpectedly, even harms syntactic correctness. This highlights that coarse-grained alignment of measurement distributions is essential for effective reward shaping. **Expectation value (EV) safeguards hard cases.** Disabling EV reduces SREV as expected and slightly degrades other semantic metrics. When distributional divergence is high, EV provides a task-specific signal that rescues otherwise borderline QASMs. **Optimization progress (Opt) provides incremental gains.** Removing Opt leads to a notable drop in HQCR, with a larger gap at Pass@10 than at Pass@1.

Tab. 1: VISTA's Pass@K and comparison with existing techniques.

Methods	Pass@1				Pass@10			
	SCR ↑	SREV ↑	RE ↓	HQCR ↑	SCR ↑	SREV ↑	RE ↓	HQCR ↑
Few-Shot Prompting								
DeepSeek-V3	94.83%	12.24%	19.20	10.00%	98.97%	26.38%	16.39	16.38%
GPT-5	87.07%	10.00%	19.94	6.90%	90.52%	27.07%	11.57	16.55%
GPT-4o	87.93%	9.83%	19.42	6.38%	88.79%	18.62%	14.08	12.07%
Post-Training (Qwen3-4B)								
Agent-Q	97.41%	18.97%	12.74	15.17%	99.65%	31.55%	10.81	23.62%
Cold Start GRPO	84.48%	19.84%	14.32	12.41%	95.17%	27.59%	11.38	18.96%
VISTA (ours)	**99.31%**	**22.41%**	**11.61**	**17.24%**	**100%**	**33.10%**	**8.48**	**27.24%**

This suggests that rewarding fewer optimization steps can also benefit the training. **Qubit mismatch penalty (QMP) ensures stability.** Without QMP, qubit-count inconsistencies increase and reward noise emerges (e.g., evaluation failures or padded comparisons), lowering all metrics. **Validity-only rewards are insufficient.** A reward that enforces only basic validity achieves reasonable SCR but lags considerably on SREV, RE, and HQCR, even performing slightly worse than Agent-Q. This underscores the necessity of semantically informed reward signals.



(a) (b)

Fig. 3: (a) HQCR with varying threshold. (b) ΔE for valid QASMs.

7.4 Training Dynamics and Stability

[TODO: TODO (Missing experiment): Training stability / learning dynamics. Add a curve of SCR/HQCR (or reward) versus training steps comparing VISTA to RL-only cold start and a non-hierarchical or "w/o RE" ablation to show that hierarchical verified rewards prevent collapse. Report at least 3 seeds or include variance bands if available.]

7.5 Verifier Efficiency: Cost Savings from Gated Evaluation

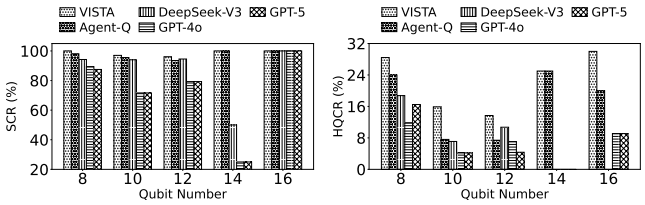
[TODO: TODO (Missing experiment): Quantify verifier-cost savings from gated evaluation. Report the fraction of samples filtered at each stage (syntax → JS → EV → Opt), the resulting number of simulator/optimizer invocations avoided, and the net wall-clock speedup/throughput gain. Include a comparison against a "no-gating" variant under (i) fixed quality or (ii) fixed compute budget.]

7.6 Robustness to Scale: Qubit Count and Circuit Complexity

In this section, we examine the scalability of VISTA with respect to the number of qubits, which directly determines the size of the underlying Hilbert space and the computational cost of quantum verification through simulation. We analyze how both the syntactic and semantic correctness of the generated quantum circuits evolve as the problem size increases.

As shown in Figure 4a, the qubit count has little impact on the syntactic correctness of VISTA and Agent-Q. In contrast, when the number of qubits reaches 14, industrial LLMs with few-shot prompting exhibit a sharp increase in compilation errors. Furthermore, Figure 4b indicates that the semantic correctness (HQCR) of industrial LLMs with few-shot learning degrades rapidly, dropping to nearly zero at larger qubit sizes.

Across both metrics, VISTA consistently outperforms all baselines, demonstrating strong robustness as the problem scale increases.



(a) (b)

Fig. 4: (a) SCR with varying qubit number. (b) HQCR with varying qubit number.

7.7 Deployment on Real Quantum Hardware

In this section, we evaluate how VISTA behaves on real quantum hardware. Both Ground-truth and VISTA circuits are compiled under an identical *same-device* constraint to ensure a fair comparison. Specifically, we transpile each circuit using the same Qiskit compilation pipeline (identical optimization level, layout and routing methods, and a fixed transpiler seed) and map them onto the same IBM quantum device. We use the IBM Torino quantum computer (`ibm_torino`) to obtain the native gate set and hardware connectivity. For each instance with n qubits, we restrict compilation to an n-qubit subdevice (physical qubits $0, \ldots, n-1$) so that both circuits

Tab. 2: Reward ablation for VISTA: contribution of each component.

Variant	Components	Pass@1				Pass@10			
		SCR ↑	SREV ↑	RE ↓	HQCR ↑	SCR ↑	SREV ↑	RE ↓	HQCR ↑
Full (VISTA)	**Val + RE + EV + Opt + QMP**	**99.31%**	**22.41%**	**11.61**	**17.24%**	**100%**	**33.10%**	**8.48**	**27.24%**
w/o EV term	Val + RE + Opt + QMP	98.62%	20.69%	11.82	16.38%	100%	31.03%	9.12	26.72%
w/o RE term	Val + EV + Opt + QMP	66.38%	5.17%	24.67	5.69%	79.82%	15.52%	18.26	16.90%
w/o Opt term	Val + RE + EV + QMP	98.79%	21.90%	11.98	16.90%	100%	32.76%	9.01	26.55%
w/o QMP	Val + RE + EV + Opt	98.79%	21.72%	12.02	16.21%	100%	31.55%	9.48	27.06%
Validity only	Val	99.13%	18.79%	12.89	14.66%	100%	30.86%	11.27	23.27%

face identical coupling constraints without inflating circuit width to the full device size.

Importantly, the Ground-truth circuits as specified in subsubsection 7.1.1 serve as solution-quality references constructed via optimized variational algorithms, but they are not guaranteed to be globally optimal in terms of circuit depth, gate count, or hardware efficiency. Therefore, improvements observed after compilation reflect hardware-aware optimization rather than deviations from a circuit-minimal baseline.

Optimality-gap comparison. We plot the optimality gap $g = E - E^*$ in Figure 5a, where E is the expectation value of the instance cost Hamiltonian estimated from 2000-shot execution on real quantum hardware after compiling both circuits with the identical transpilation pipeline and mapping them to `ibm_torino`. E^* denotes the minimum eigenvalue of the Hamiltonian provided in the dataset. The dashed diagonal indicates $g_{\text{VISTA}} = g_{\text{GT}}$.

As shown in Figure 5a, the points cluster near the diagonal, indicating strong agreement between the optimality gaps of VISTA and Ground-truth circuits on real hardware. This confirms that VISTA-generated circuits maintain comparable optimization performance to the reference circuits under realistic hardware execution conditions.

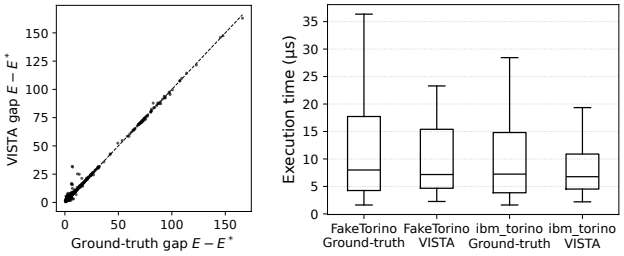
Execution-time comparison. Beyond solution quality, we evaluate the hardware-level execution cost of generated circuits. For each of the $K = 50$ evaluation instances, we compile both Ground-truth and VISTA circuits using the same-device compilation pipeline, and target both the noise-calibrated simulator `FakeTorino` and the real IBM backend `ibm_torino`.

To estimate physical execution cost independent of queueing delay and classical control overhead, we use the scheduled circuit duration reported after transpilation. Execution time is computed as the circuit's scheduled duration using backend instruction timings and is reported in microseconds.

Figure 5b shows the distribution of scheduled execution time across instances for both methods and backends. Across both `FakeTorino` and `ibm_torino`, VISTA circuits consistently exhibit shorter scheduled durations than Ground-truth circuits, indicating improved hardware efficiency after compilation. Using the per-instance speedup

$$S = T_{\text{GT}}/T_{\text{VISTA}},$$

computed from scheduled duration, VISTA achieves an average speedup of 1.97× on `ibm_torino` and 2.12× on `FakeTorino`.



(a) (b)

Fig. 5: (a) Optimality-gap agreement between Ground-truth and VISTA under a shared hardware model. (b) Physical execution-time comparison (μs) for Ground-truth and VISTA circuits on `FakeTorino` (noise-calibrated simulation) and `ibm_torino` (IBM Quantum hardware).

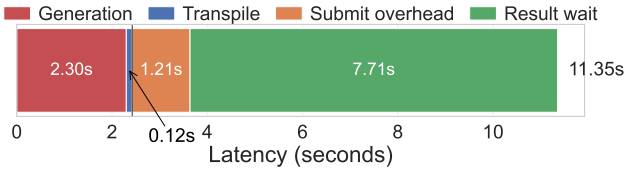


Fig. 6: End-to-end latency breakdown for running ≤ 5-qubit circuits on Helmi (VTT Q5) in our quantum agentic system (Lumi as the classical host). The stacked bar shows the mean latency per circuit, decomposed into generation time (2.3 s), transpilation, submission overhead, and backend result wait (execution + result fetch), averaged over successfully submitted jobs.

Interestingly, the generated circuits can be more hardware-efficient than the offline ground-truth circuits after transpilation. This suggests that VISTA may discover circuit structures that compile to shorter scheduled durations under hardware constraints, even when the reference circuits are semantically optimal for the underlying optimization problem.

[TODO: SQUEEZE: Move the detailed latency breakdown table/figure to Appendix; keep 1 sentence headline in main (e.g., end-to-end dominated by X).]

Classical-Quantum Pipeline Latency Breakdown. Figure 6 presents the end-to-end latency breakdown of our hybrid classical-quantum pipeline when executing ≤5 qubit circuits on Helmi (VTT Q5), with Lumi serving as the classical host. The total latency is decomposed into four stages: *Generation*, the LLM inference time required to produce the OpenQASM 3.0 circuit from a natural-language prompt

(2.30 s on average); *Transpile*, the client-side compilation time spent translating the generated circuit into the backend-native representation, including gate decomposition, device mapping under connectivity constraints, routing, optimization passes, and construction of the execution payload (0.12 s); *Submit overhead*, the time required to transmit the job to the backend excluding execution, including authentication, serialization, network round trips, and job acknowledgement (1.21 s); and *Result wait*, the time from successful submission until results are returned, covering queueing, physical execution (or calibrated simulation), and provider-side post-processing as observed from the client (7.71 s).

8 Limitations and Broader Applicability

[TODO: SQUEEZE: Keep 4–6 concise limitations + applicability bullets in main. Move extended discussion to Appendix if needed.]

Limitations. **[TODO: Summarize limitations already implied by the paper: dependence on an expensive verifier (simulation/optimization), reliance on task-specific semantic metrics, and scalability constraints as circuit size increases.]**

Broader applicability. **[TODO: Briefly state how the verifier-in-the-loop + hierarchical reward + gated evaluation blueprint transfers to other domains where semantic correctness is externally checkable but expensive (e.g., compilers, simulators, solvers).]**

9 Related Work

In recent years, there has been increasing interest in applying GPT- and LLM-based tools to address challenges in designing and constructing circuits for various quantum computational challenges. In one of the first works, [18] fine-tuned large language models to serve as a Qiskit Code Assistant, which was evaluated with [41]. This work was further improved with post-training reinforcement learning [11]. [7] leveraged Chain-of-Thought (CoT) reasoning and Retrieval-Augmented Generation (RAG) in a multi-agent setting to facilitate the synthesis of quantum circuits. These efforts are closely tied to the Qiskit framework, whereas our approach targets OpenQASM 3.0 [8], a platform-independent standard now supported by Qiskit, PennyLane, and Cirq. LLMs have also been used for optimizing ansatz design [26, 40], and [5] used language models to design quantum experiments. [16] utilized LLMs for guided ansatz design in financial modeling.

Compared to LLM-based methods, standard transformers and GPT-based systems have received more attention. [14] trained the standard GPT model to predict measurement outcomes from a neutral atom quantum computer. The findings revealed limitations in the standard GPT model's ability to predict measurement outcomes, which could prove valuable in expanding our knowledge of the boundaries of present-day LLMs. Using the standard transformer-based approach, Nvidia has designed an optimization pipeline that produces quantum circuits, which are aimed at identifying ground states in electronic structure Hamiltonians [31]. Since the trainable parameters are in the transformer model, the method tries to circumvent some of the problems in current variational methods, like barren plateaus. [4] developed KetGPT, which uses a GPT-based model to generate realistic quantum circuits. The model is trained in QASMBench circuits [25]. The circuits are restricted to the OpenQASM 2.0 format, without supporting parameters. Finally, [39] developed QAOA-GPT, which is capable of generating QAOA-type circuits that solve the MaxCut problem. [12] is the first to use a quantum-verifiable reward to post-training LLMs to generate Qiskit code.

10 Conclusion

We presented VISTA, a verifier-in-the-loop, tool-augmented agentic RL system for semantic program synthesis, with OpenQASM 3.0 quantum circuit generation as a successful application. The paper makes three core contributions. First, we contribute the core system: verifier-in-the-loop, tool-augmented agentic RL system for semantic program synthesis. Second, we contribute optimization technique I: hierarchical verified reward optimization, which provides stable, task-aligned learning signals from executability through objective-level quality. Third, we contribute optimization technique II: cost-aware gated evaluation optimization, which reduces verifier overhead by prioritizing low-cost checks and reserving expensive evaluations for promising candidates.

Empirically, VISTA consistently outperforms prompting-based industrial LLMs and SFT-only and RL-only baselines on both executability and semantic quality. Overall, the results show that combining hierarchical reward shaping with gated verification is an effective optimization strategy for AI agent systems in semantics-heavy generation tasks, with quantum software as a successful application domain.

References

[1] [n. d.]. Amazon Braket Pricing. https://aws.amazon.com/braket/pricing/. Accessed 2026-02-23.

[2] [n. d.]. IBM Quantum: Products and services (Quantum compute access plans). https://www.ibm.com/quantum/products. Accessed 2026-02-23.

[3] Amira Abbas, Andris Ambainis, Brandon Augustino, Andreas Bärtschi, Harry Buhrman, Carleton Coffrin, Giorgio Cortiana, et al. 2024. Challenges and opportunities in quantum optimization. *Nature Reviews Physics* 6, 12 (Dec. 2024), 718–735. doi:10.1038/s42254-024-00770-9 Publisher: Nature Publishing Group.

[4] Boran Apak, Medina Bandic, Aritra Sarkar, and Sebastian Feld. 2024. KetGPT – Dataset Augmentation of Quantum Circuits Using Transformers. In *Computational Science – ICCS 2024*, Leonardo Franco, Clélia de Mulatier, Maciej Paszynski, Valeria V. Krzhizhanovskaya, Jack J. Dongarra, and Peter M. A. Sloot (Eds.). Springer Nature Switzerland, Cham, 235–251.

[5] Sören Arlt, Haonan Duan, Felix Li, Sang Michael Xie, Yuhuai Wu, and Mario Krenn. 2024. Meta-Designing Quantum Experiments with Language Models. *arXiv preprint arXiv:2406.02470* (2024).

[6] Endre Boros and Peter L. Hammer. 2002. Pseudo-Boolean optimization. *Discrete Applied Mathematics* 123, 1 (2002), 155–225. doi:10.1016/S0166-218X(01)00341-9

[7] Charlie Campbell, Hao Mark Chen, Wayne Luk, and Hongxiang Fan. 2025. Enhancing LLM-based Quantum Code Generation with Multi-Agent Optimization and Quantum Error Correction. *arXiv preprint arXiv:2504.14557* (2025).

[8] Andrew W. Cross, Ali Javadi-Abhari, Thomas Alexander, Niel de Beaudrap, Lev S. Bishop, et al. 2022. OpenQASM 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing* 3, 3 (Sept. 2022), 1–50. doi:10.1145/3505636 arXiv:2104.14722 [quant-ph].

[9] DeepSeek-AI, Daya Guo, and et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948* (2025). doi:10.48550/arXiv.2501.12948

[10] Cirq Developers. 2025. *Cirq*. Zenodo. doi:10.5281/ZENODO.4062499

[11] Nicolas Dupuis, Adarsh Tiwari, Youssef Mroueh, David Kremer, Ismael Faro, and Juan Cruz-Benito. 2025. Quantum Verifiable Rewards for Post-Training Qiskit Code Assistant. arXiv:2508.20907 (Aug. 2025). doi:10.48550/arXiv.2508.20907 arXiv:2508.20907 [quant-ph].

[12] Nicolas Dupuis, Adarsh Tiwari, Youssef Mroueh, David Kremer, Ismael Faro, and Juan Cruz-Benito. 2025. Quantum Verifiable Rewards for Post-Training Qiskit Code Assistant. *arXiv preprint arXiv:2508.20907* (2025).

[13] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A Quantum Approximate Optimization Algorithm. *arXiv preprint arXiv:1411.4028* (2014). doi:10.48550/arXiv.1411.4028

[14] David Fitzek, Yi Hong Teoh, Hin Pok Fung, Gebremedhin A. Dagnew, Ejaaz Merali, M. Schuyler Moss, Benjamin MacLellan, and Roger G. Melko. 2024. RydbergGPT. *arXiv preprint arXiv:2405.21052* (may 2024). doi:10.48550/arXiv.2405.21052

[15] Harper R. Grimsley, Sophia E. Economou, Edwin Barnes, and Nicholas J. Mayhall. 2019. An adaptive variational algorithm for exact molecular simulations on a quantum computer. *Nature Communications* 10, 1 (July 2019), 3007. doi:10.1038/s41467-019-10988-2

[16] Yaswitha Gujju, Romain Harang, and Tetsuo Shibuya. 2025. LLM-Guided Ansätze Design for Quantum Circuit Born Machines in Financial Generative Modeling. arXiv:2509.08385 [quant-ph]

[17] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. GPT-4o System Card. *arXiv preprint arXiv:2410.21276* (2024).

[18] IBM Quantum. 2025. Qiskit Code Assistant. https://quantum.cloud.ibm.com/docs/en/guides/qiskit-code-assistant Accessed: 2025-07-11.

[19] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum computing with Qiskit. arXiv:2405.08810 [quant-ph]

[20] Linus Jern, Valter Uotila, Cong Yu, and Bo Zhao. 2025. Agent-Q: Fine-Tuning Large Language Models for Quantum Circuit Generation and Optimization. In *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE.

[21] Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, et al. 2025. VerlTool: Towards Holistic Agentic Reinforcement Learning with Tool Use. *arXiv preprint arXiv:2509.01055* (2025).

[22] Richard M. Karp. 1972. *Reducibility among Combinatorial Problems.* Springer US, Boston, MA, 85–103. doi:10.1007/978-1-4684-2001-2_9

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[24] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. *arXiv preprint arXiv:2207.01780* (2022). arXiv:2207.01780 [cs.LG] doi:10.48550/arXiv.2207.01780

[25] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. 2023. QASMBench: A Low-Level Quantum Benchmark Suite for NISQ Evaluation and Simulation. *ACM Transactions on Quantum Computing* 4, 2, Article 10 (Feb. 2023), 26 pages. doi:10.1145/3550488

[26] Zhiding Liang, Jinglei Cheng, Rui Yang, Hang Ren, Zhixin Song, Di Wu, Xuehai Qian, Tongyang Li, and Yiyu Shi. 2023. Unleashing the Potential of LLMs for Quantum Computing: A Study in Quantum Architecture Design. arXiv:2307.08191 (July 2023). doi:10.48550/arXiv.2307.08191 arXiv:2307.08191 [quant-ph].

[27] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024).

[28] Ilya Loshchilov and Frank Hutter. 2017. Decoupled Weight Decay Regularization. *arXiv preprint arXiv:1711.05101* (2017).

[29] Andrew Lucas. 2014. Ising formulations of many NP problems. *Frontiers in Physics* 2 (Feb. 2014). doi:10.3389/fphy.2014.00005

[30] Microsoft. [n. d.]. *Azure Quantum Development Kit.* https://github.com/microsoft/qsharp

[31] Kouhei Nakaji, Lasse Bjørn Kristensen, Jorge A. Campos-Gonzalez-Angulo, Mohammad Ghazi Vakili, Haozhe Huang, Mohsen Bagherimehrab, Christoph Gorgulla, FuTe Wong, Alex McCaskey, Jin-Sung Kim, Thien Nguyen, Pooja Rao, and Alan Aspuru-Guzik. 2024. The generative quantum eigensolver (GQE) and its application for ground state search. *arXiv preprint arXiv:2401.09253* (jan 2024). doi:10.48550/arXiv.2401.09253

[32] Michael A. Nielsen and Isaac L. Chuang. 2010. *Quantum computation and quantum information* (10th anniversary ed ed.). Cambridge University Press, Cambridge ; New York.

[33] OpenAI. 2025. *Introducing GPT-5.* https://openai.com/index/introducing-gpt-5/ Accessed: 2025-09-21.

[34] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O'Brien. 2014. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications* 5 (July 2014), 4213. doi:10.1038/ncomms5213

[35] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. 2025. ToolRL: Reward is All Tool Learning Needs. arXiv:2504.13958 [cs.LG]

[36] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).

[37] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366* (2023). arXiv:2303.11366 [cs.AI] doi:10.48550/arXiv.2303.11366

[38] Robert S. Smith, Michael J. Curtis, and William J. Zeng. 2017. A Practical Quantum Instruction Set Architecture. arXiv:1608.03355 [quant-ph]

[39] Ilya Tyagin, Marwa H. Farag, Kyle Sherbert, Karunya Shirali, Yuri Alexeev, and Ilya Safro. 2025. QAOA-GPT: Efficient Generation of Adaptive and Regular Quantum Approximate Optimization Algorithm Circuits. arXiv:2504.16350 (April 2025). doi:10.48550/arXiv.2504.16350 arXiv:2504.16350 [quant-ph].

[40] Kento Ueda and Atsushi Matsuo. 2025. Optimizing Ansatz Design in Quantum Generative Adversarial Networks Using Large Language Models. arXiv:2503.12884 (March 2025). doi:10.48550/arXiv.2503.12884 arXiv:2503.12884 [quant-ph].

[41] Sanjay Vishwakarma, Francis Harkins, Siddharth Golecha, Vishal Sharathchandra Bajpe, Nicolas Dupuis, Luca Buratti, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. 2024. Qiskit HumanEval: An Evaluation Benchmark For Quantum Code Generative Models. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1. IEEE, 1169–1176.

[42] volcengine. 2025. verl: Volcano Engine Reinforcement Learning for LLMs. https://github.com/volcengine/verl. Accessed: 2025-12-27.

[43] Zebo Yang, Maede Zolanvari, and Raj Jain. 2023. A Survey of Important Issues in Quantum Computing and Communications. *IEEE Communications Surveys & Tutorials* 25, 2 (2023), 1059–1094. doi:10.1109/COMST.2023.3254481

[44] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, et al. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. arXiv:2304.11277 [cs.DC]

A Additional Details

A.1 Training Data, Ground Truth, and Hyperparameters

Training data. We utilize the training data from [20], which is one of the most extensive available datasets of quantum circuits in QASM format covering 12 central optimization problem primitives on graphs [22], and many of their abstract descriptions appeared in [29]. The full dataset contains 13,914 samples, which we split into training and validation sets using a 96/4 ratio. The key characteristics are parameterized circuits with optimized parameters, problem Hamiltonians, and the smallest/largest eigenvalues for the Hamiltonians. The dataset is constructed by generating random graph instances and solving each instance using QAOA, VQE, or adaptive VQE under noiseless simulation. We retain only instances for which optimization converges.

Ground-truth circuits and reward reference. The ground-truth OpenQASM 3.0 circuits are constructed offline following the Agent-Q data generation methodology [20]. Each optimization instance is encoded as a QUBO/HUBO cost Hamiltonian and solved using variational quantum algorithms. Circuit parameters are optimized under noiseless simulation until the most probable measurement outcome matches the exact solution from a classical eigensolver. Only successfully converged circuits are retained and treated as references. During reinforcement learning, these references serve as reward baselines, and associated metadata (Hamiltonians and exact eigenvalues) support expectation-value-based reward computation.

Training settings. We fine-tune a 4B model with GRPO on 16×H100-64GB GPUs using FSDP [44]. Each prompt samples n=16 rollouts via vLLM [23] (temperature 0.7, top-p = 0.8). The average training time is 48 hours.

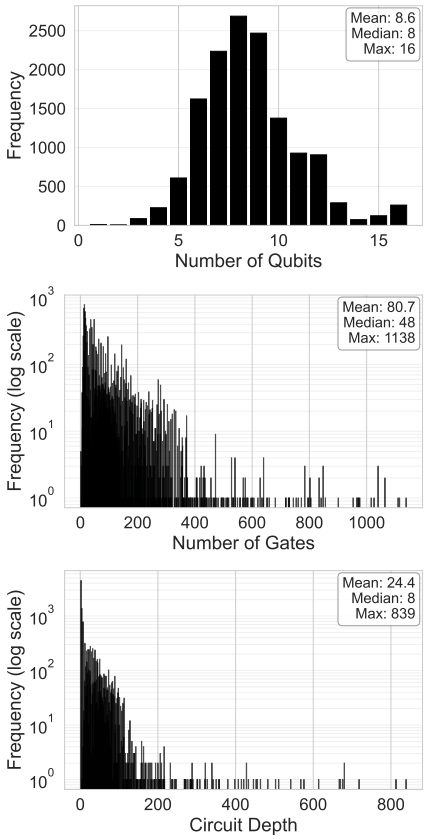


Fig. 7: Dataset statistics: distribution of (a) number of qubits, (b) number of gates, and (c) circuit depth.

Tab. 3: Training settings for tool-augmented agentic RL with a 4B SFT model.

Component	Configuration
Base model	Qwen3-4B-Instruct-2507
SFT setup	Quantum Datasets [20]
SFT learning rate	2×10^{-5}
Rollout Num	16
Temperature	0.7
top_p	0.8
top_k	-1
Token limits	Prompt: 1024; Response: 9216; Observation: 2048; Action: 8192
Optimizer	AdamW [28]
Learning rate	1×10^{-6}
Epochs	10
Batch size	128

A.2 Evaluation Placeholders for Camera-Ready

A.3 Materials Moved from Main for the 9-Page Limit

[TODO: SQUEEZE: Place extended content here to keep the main paper within 9 pages. Suggested items: (i) full training and rollout hyperparameters, (ii) detailed baseline prompting setups, (iii) full gating cost breakdown (stage pass rates, verifier time), (iv) training dynamics over steps/seeds, (v) per-primitive breakdown tables/plots, (vi) detailed real-device latency breakdown.]

Extended evaluation setup details.

Tab. 4: Verifier-cost breakdown by gated evaluation stage (placeholder). Report: (i) fraction of samples reaching each stage, (ii) average simulator/optimizer time per prompt, and (iii) total verifier time per training step. Include a "no-gating" variant under matched quality or matched compute.

TODO: Fill gating cost table.

Fig. 8: Training dynamics (placeholder). Plot SCR/HQCR (or reward) versus training steps comparing VISTA to RL-only cold start and a non-hierarchical (or w/o RE) variant, with ≥3 seeds or variance bands.

TODO: Insert training-dynamics plot.

Fig. 9: Per-primitive breakdown (placeholder). Report HQCR or SREV per optimization primitive (12 tasks). Summarize key take-aways in the main text and place the full breakdown here.

TODO: Insert per-primitive breakdown.

Tab. 5: Real-device execution details (placeholder). Report qubit count and transpiled depth distributions, number of instances, shots, and (if available) variance/error bars across runs.

TODO: Fill device-details table.

[TODO: MOVE: Insert full dataset construction / split / hyperparameters here.]

Extended baseline configurations.

[TODO: MOVE: Insert full prompting templates / few-shot examples and API details here.]

Detailed pipeline latency breakdown.

[TODO: MOVE: Insert full Helmi/VTT Q5/LUMI latency table/figure here.]